

VAST Admin Training

Complete Lab Documentation — Verified

Modules 1–9 | Commands • Purpose • Diagrams • Comments

Ubuntu 24.04 Live-Verified | Qualcomm Delivery

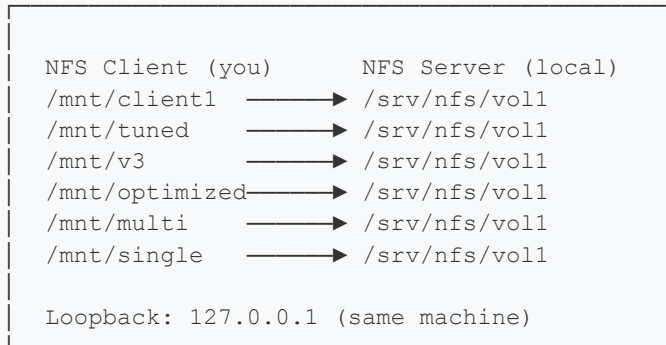
August 2026

PRE-REQUISITE SETUP — Run Once Before All Modules

🎯 **PURPOSE:** Install all tools and configure NFS server. Every module depends on this setup.

Architecture: What We Are Building

Your Ubuntu 24.04 Laptop



This simulates a VAST cluster without real hardware.
On real VAST: replace 127.0.0.1 with cluster IP.

Step 1: Install All Tools

🎯 **PURPOSE:** Install all 24 tools needed across Modules 1–9 in one command.

```
# Update package list first
sudo apt update

# Install all tools in one command — takes 3-5 minutes
# nfs-common          = nfsstat, showmount, mountstats, nfsiostat
# nfs-kernel-server   = local NFS server for lab
# lvm2                = LVM commands: pvcreate, vgcreate, lvcreate (Module 1)
# sysstat             = iostat, sar, mpstat (Modules 2,3,7)
# fio                 = storage benchmark (Modules 2,3,5,7)
# iperf3              = network bandwidth test (Module 3)
# hping3              = advanced ping (Module 3)
# iproute2            = tc traffic control — needed for netem (Modules 3,7)
# nmap                = port scanner (Module 6)
# dstat               = combined disk+network stats (Module 2)
sudo apt install -y nfs-common nfs-kernel-server lvm2 sysstat iotop fio \
  iperf3 net-tools ethtool hping3 nmap traceroute nload irqbalance \
  cifs-utils smbclient nvme-cli lshw bc tree curl wget dstat
```

✅ **EXPECTED OUTPUT:** Processing triggers for libc-bin ... (no errors at end)

Step 2: Verify All Tools Installed

🎯 **PURPOSE:** Confirm every tool is available before starting any module.

```
# Check each tool — prints ✓ if found, ✗ MISSING if not
for tool in fio iperf3 iostat iotop dstat nfsstat nfsiostat mountstats \
  showmount rpcinfo nvme lsblk lshw ethtool nmap traceroute tc ss \
  netstat mpstat vmstat sar hping3 nload irqbalance; do
  which $tool > /dev/null 2>&1 && echo "✓ $tool" || echo "✗ MISSING: $tool"
done
```

✓ EXPECTED OUTPUT: ✓ fio ✓ iperf3 ✓ iostat ... (all 23 tools show ✓)

Step 3: Enable sysstat

🎯 PURPOSE: sysstat is disabled by default on Ubuntu 24.04. Enable it for sar and iostat.

```
# Change ENABLED=false to ENABLED=true in sysstat config file
sudo sed -i 's/ENABLED="false"/ENABLED="true"/' /etc/default/sysstat

# Enable on boot and start now
sudo systemctl enable sysstat && sudo systemctl start sysstat

# Confirm running
systemctl is-active sysstat
```

✓ EXPECTED OUTPUT: active

Step 4: Create NFS Server Directories

🎯 PURPOSE: 4 folders simulate different VAST volumes — AI data, scratch, archive, and main volume.

```
/srv/nfs/
├── voll/      ← Main training volume (read-write)
├── aidata/    ← Simulates AI model storage (read-write)
├── scratch/   ← Simulates HPC scratch space (read-write)
└── archive/   ← Simulates read-only archive (read-only)
```

```
# Create all 4 server directories in one command
sudo mkdir -p /srv/nfs/voll /srv/nfs/aidata /srv/nfs/scratch /srv/nfs/archive

# Give current user ownership so writes work without sudo
sudo chown $USER:$USER /srv/nfs/voll /srv/nfs/aidata /srv/nfs/scratch

# Set rwxr-xr-x permissions
sudo chmod 755 /srv/nfs/voll /srv/nfs/aidata /srv/nfs/scratch

echo "Directories created"
```

✓ EXPECTED OUTPUT: Directories created

Step 5: Configure NFS Exports

🎯 PURPOSE: Tell NFS server which folders to share and who can access them.

```
# Add each export individually - one line per folder
# rw          = allow read and write
# sync        = write to disk before confirming to client
# no_subtree_check = skip subtree checking (better performance)
# no_root_squash = allow root access (lab only - not for production)
sudo bash -c 'echo "/srv/nfs/voll
127.0.0.1(rw,sync,no_subtree_check,no_root_squash)" >> /etc/exports'
sudo bash -c 'echo "/srv/nfs/aidata
127.0.0.1(rw,sync,no_subtree_check,no_root_squash)" >> /etc/exports'
sudo bash -c 'echo "/srv/nfs/scratch
127.0.0.1(rw,sync,no_subtree_check,no_root_squash)" >> /etc/exports'
# archive is read-only - no no_root_squash needed
```

```
sudo bash -c 'echo "/srv/nfs/archive 127.0.0.1(ro, sync, no_subtree_check)" >> /etc/exports'

# Verify exports are written correctly
cat /etc/exports | grep srv
```

✓ EXPECTED OUTPUT: /srv/nfs/vol1 127.0.0.1(rw,...) — 4 lines

Step 6: Start NFS Server

```
# Enable nfs-kernel-server to start on boot
sudo systemctl enable nfs-kernel-server
sudo systemctl start nfs-kernel-server
sudo systemctl enable rpcbind
sudo systemctl start rpcbind

# Apply the exports – must run after any /etc/exports change
sudo exportfs -ra

# Confirm server is exporting correctly
showmount -e 127.0.0.1
```

✓ EXPECTED OUTPUT: Export list for 127.0.0.1: /srv/nfs/archive /srv/nfs/scratch /srv/nfs/aidata /srv/nfs/vol1

Step 7: Create All Mount Points

🎯 **PURPOSE:** Each mount point is a local folder where the remote NFS share appears. Must exist before mounting.

```
# Create all 14 mount points used across all 9 modules
# -p = create parent dirs too, no error if already exists
for mnt in data client1 vol1 v3 v4 baseline tuned optimized multi single \
    vast-data vast-scratch vast-archive lvmvol; do
    sudo mkdir -p /mnt/$mnt && echo "✓ /mnt/$mnt"
done
```

✓ EXPECTED OUTPUT: ✓ /mnt/data ✓ /mnt/client1 ... ✓ /mnt/lvmvol (14 lines)

Step 8: Mount All NFS Exports

🎯 **PURPOSE:** Each mount uses different options to demonstrate performance differences in later modules.

Mount Point	Key Options	Used In
/mnt/client1	default	All modules — main working mount
/mnt/vol1	default	Module 1 LVM comparison
/mnt/v3	vers=3 (NFSv3)	Module 4 protocol comparison
/mnt/v4	vers=4 (NFSv4)	Module 4 protocol comparison
/mnt/baseline	default (untuned)	Module 5 baseline measurement
/mnt/tuned	nconnect=8, rsize/wsize=1MB	Module 5 tuned comparison
/mnt/optimized	nconnect=16, noatime	Module 5 optimized comparison
/mnt/multi	nconnect=8	Module 3 multi-connection test
/mnt/single	nconnect=1	Module 3 single-connection test

```
# Standard mounts – default NFS options
sudo mount -t nfs 127.0.0.1:/srv/nfs/vol1 /mnt/client1
sudo mount -t nfs 127.0.0.1:/srv/nfs/vol1 /mnt/vol1
```

```
# Protocol version mounts – for Module 4
sudo mount -t nfs -o vers=3 127.0.0.1:/srv/nfs/vol1 /mnt/v3
sudo mount -t nfs -o vers=4 127.0.0.1:/srv/nfs/vol1 /mnt/v4

# Baseline – untuned, for comparison in Module 5
sudo mount -t nfs 127.0.0.1:/srv/nfs/vol1 /mnt/baseline

# Tuned: nconnect=8 opens 8 TCP connections
# rsize/wsize=1048576 = 1MB read/write block size
sudo mount -t nfs -o nconnect=8,rsize=1048576,wsiz=1048576 \
  127.0.0.1:/srv/nfs/vol1 /mnt/tuned

# Optimized: nconnect=16, noatime skips access time updates
sudo mount -t nfs \
  -o vers=3,nconnect=16,rsize=1048576,wsiz=1048576,noatime,hard,proto=tcp \
  127.0.0.1:/srv/nfs/vol1 /mnt/optimized

# nconnect comparison mounts for Module 3
sudo mount -t nfs -o vers=3,nconnect=8 127.0.0.1:/srv/nfs/vol1 /mnt/multi
sudo mount -t nfs -o vers=3,nconnect=1 127.0.0.1:/srv/nfs/vol1 /mnt/single

# Verify all mounts – use nfs,nfs4 to catch both NFSv3 and NFSv4
findmnt -t nfs,nfs4 -o TARGET,SOURCE
```

✓ EXPECTED OUTPUT: /mnt/v3 /mnt/client1 /mnt/vol1 ... (9 mounts listed)

Step 9: Fix Permissions

```
# Give current user write access to all NFS server directories
sudo chown $USER:$USER /srv/nfs/vol1 /srv/nfs/aidata /srv/nfs/scratch

# Quick write test on 3 key mounts
touch /mnt/client1/test_write && echo "✓ Write OK on client1" && rm
/mnt/client1/test_write
touch /mnt/tuned/test_write && echo "✓ Write OK on tuned" && rm
/mnt/tuned/test_write
touch /mnt/optimized/test_write && echo "✓ Write OK on optimized" && rm
/mnt/optimized/test_write
```

✓ EXPECTED OUTPUT: ✓ Write OK on client1 ✓ Write OK on tuned ✓ Write OK on optimized

Step 10: LVM Setup

🎯 PURPOSE: Simulate a physical disk using a loop device — no extra disk needed.

```
Loop device flow:
/tmp/lvm_disk.img (500MB file)
  ↓ losetup
/dev/loop15 (fake block device)
  ↓ pvcreate
Physical Volume
  ↓ vgcreate
Volume Group: vast_vg
  ↓ lvcreate
Logical Volume: vast_vg/vol1 (400MB)
  ↓ mkfs.ext4 + mount
/mnt/lvmvol
```

```
# Create a 500MB empty file to use as a fake disk
sudo dd if=/dev/zero of=/tmp/lvm_disk.img bs=1M count=500 2>&1 | tail -1

# Attach the file as a loop device (becomes /dev/loopN)
LOOP=$(sudo losetup -f --show /tmp/lvm_disk.img) && echo "Loop device: $LOOP"

# Mark loop device as LVM Physical Volume
sudo pvcreate $LOOP

# Create Volume Group named vast_vg
sudo vgcreate vast_vg $LOOP

# Create 400MB Logical Volume named voll
sudo lvcreate -L 400M -n voll vast_vg

# Format with ext4 filesystem
sudo mkfs.ext4 /dev/vast_vg/voll 2>&1 | tail -3

# Mount and set ownership
sudo mount /dev/vast_vg/voll /mnt/lvmvol
sudo chown $USER:$USER /mnt/lvmvol
df -h /mnt/lvmvol
```

✓ EXPECTED OUTPUT: /dev/mapper/vast_vg-voll 359M 24K 331M 1% /mnt/lvmvol

Step 11: Load netem Kernel Module

🎯 **PURPOSE:** netem is needed for injecting artificial network delays in Modules 3 and 7.

```
# Load the Network Emulator kernel module
sudo modprobe sch_netem && echo "✓ netem loaded"

# Confirm it is loaded (shows module name and size)
lsmod | grep sch_netem
```

✓ EXPECTED OUTPUT: sch_netem 28672 0

Daily Reset — Run Every Morning Before Training

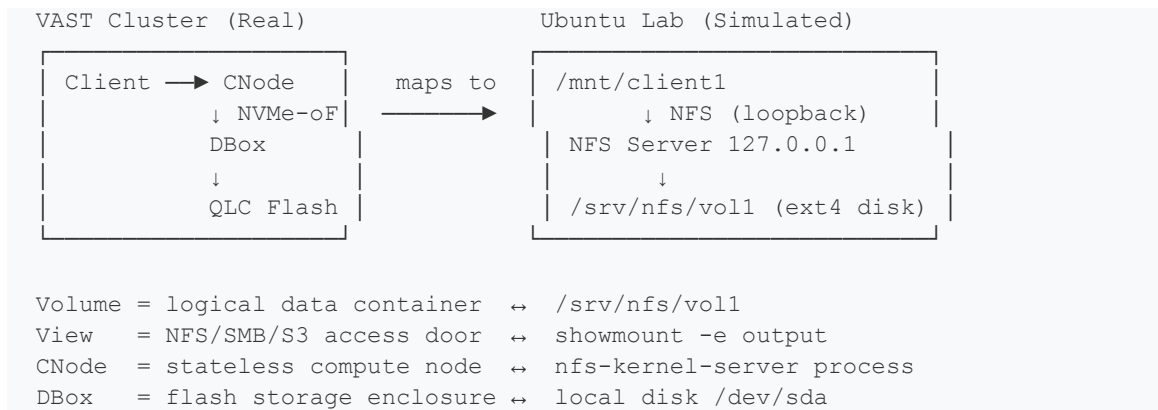
⚠ **NOTE:** After reboot, all NFS mounts disappear. Run this single command before each training day.

```
# Reload systemd, start NFS, remount all 9 mounts in one command
sudo systemctl daemon-reload && \
sudo systemctl start nfs-kernel-server && \
sudo mount -t nfs 127.0.0.1:/srv/nfs/voll /mnt/client1 && \
sudo mount -t nfs 127.0.0.1:/srv/nfs/voll /mnt/voll && \
sudo mount -t nfs -o vers=3 127.0.0.1:/srv/nfs/voll /mnt/v3 && \
sudo mount -t nfs -o vers=4 127.0.0.1:/srv/nfs/voll /mnt/v4 && \
sudo mount -t nfs 127.0.0.1:/srv/nfs/voll /mnt/baseline && \
sudo mount -t nfs -o nconnect=8,rsz=1048576,wsz=1048576 \
127.0.0.1:/srv/nfs/voll /mnt/tuned && \
sudo mount -t nfs -o \
vers=3,nconnect=16,rsz=1048576,wsz=1048576,noatime,hard,proto=tcp \
127.0.0.1:/srv/nfs/voll /mnt/optimized && \
sudo mount -t nfs -o vers=3,nconnect=8 127.0.0.1:/srv/nfs/voll /mnt/multi && \
sudo mount -t nfs -o vers=3,nconnect=1 127.0.0.1:/srv/nfs/voll /mnt/single && \
echo "ALL MOUNTS READY"
```

✓ EXPECTED OUTPUT: ALL MOUNTS READY

MODULE 1 — Storage Architecture

Architecture Diagram



Scenario 1: Map Storage Path End to End

🎯 **PURPOSE:** Verify complete path from client to storage. First health check every VAST admin runs.

```
# List all available NFS exports from the server
# Equivalent to: what volumes does the VAST cluster expose?
showmount -e 127.0.0.1
```

✅ **EXPECTED OUTPUT:** Export list for 127.0.0.1: /srv/nfs/archive /srv/nfs/scratch /srv/nfs/aidata /srv/nfs/vol1

```
# Show all mounted NFS filesystems
# TARGET = where it appears on client SOURCE = server path
findmnt -t nfs,nfs4 -o TARGET,SOURCE
```

✅ **EXPECTED OUTPUT:** /mnt/client1 127.0.0.1:/srv/nfs/vol1 (9 mounts listed)

```
# Show capacity of the mounted volume
# Size=total Used=used Avail=free Use%=percentage used
df -h /mnt/client1
```

✅ **EXPECTED OUTPUT:** 127.0.0.1:/srv/nfs/vol1 96G 18G 74G 20% /mnt/client1

```
# List contents and permissions of mount point
# Shows owner, group, permissions, timestamps
ls -la /mnt/client1
```

✅ **EXPECTED OUTPUT:** drwxr-xr-x 2 user user 4096 Aug 8 12:36 .

Scenario 2: Multi-Volume Layout

🎯 **PURPOSE:** Verify all volumes accessible and writable. Simulates VAST multi-volume architecture.

```
# List all server-side volumes
ls /srv/nfs/
```

✅ **EXPECTED OUTPUT:** aidata archive scratch vol1

```
# Write test files to each volume
```



```
# && means: only run echo if touch succeeds
touch /srv/nfs/aidata/ai_model.bin && echo "aidata write OK"
touch /srv/nfs/scratch/temp.dat && echo "scratch write OK"

# Verify files exist
ls /srv/nfs/aidata/
ls /srv/nfs/scratch/

# Confirm server exports
showmount -e 127.0.0.1
```

✓ EXPECTED OUTPUT: aidata write OK | scratch write OK | ai_model.bin | temp.dat | 4 exports listed

Distributed System Challenge: CNode Failover

🎯 **PURPOSE:** Prove hard NFS mount recovers automatically when server restarts — simulating VAST CNode failover.

```
T=0s   Terminal 1: dd starts writing 200MB
T=3s   Terminal 2: NFS server STOPS (CNode down)
T=8s   Terminal 2: NFS server STARTS (new CNode up)
T=?s   Terminal 1: dd completes — 200MB file intact
```

```
Hard mount: client waits and retries → recovers
Soft mount: client returns error immediately → fails
```

TERMINAL 1 — Start writing:

```
# Write 200MB — will pause when server stops, resume after restart
dd if=/dev/zero of=/mnt/client1/bigfile bs=1M count=200
```

TERMINAL 2 — Simulate failure and recovery (run immediately after Terminal 1):

```
# Wait 3s, stop NFS server, wait 5s, restart NFS server
sleep 3 && sudo systemctl stop nfs-kernel-server && echo "SERVER STOPPED" \
    && sleep 5 && sudo systemctl start nfs-kernel-server && echo "SERVER RESTARTED"
```

TERMINAL 1 — After dd completes:

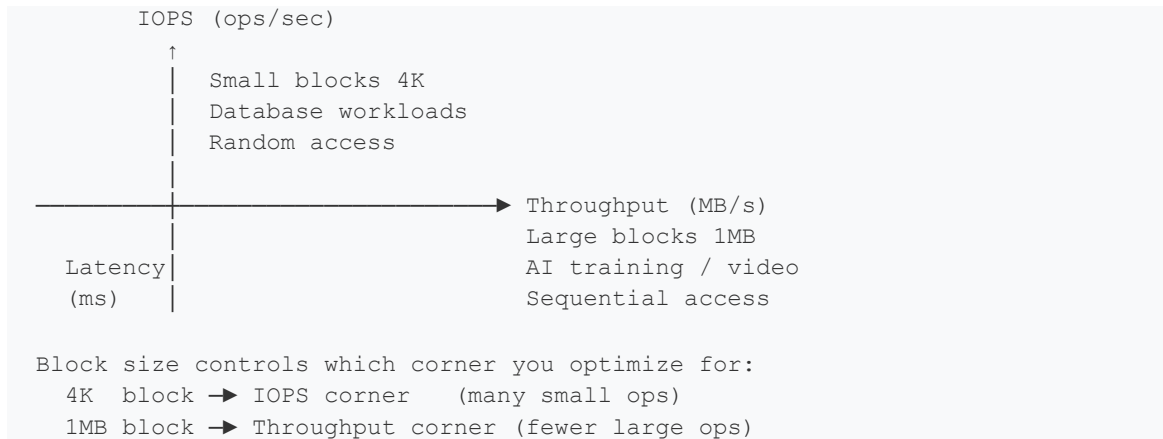
```
# Verify 200MB file is intact after server recovery
ls -lh /mnt/client1/bigfile
echo "Recovery complete"
rm -f /mnt/client1/bigfile
```

✓ EXPECTED OUTPUT: SERVER STOPPED | SERVER RESTARTED | 200M file intact | Recovery complete

💡 **KEY LESSON:** Hard mount = client waits and recovers. VAST CNodes are stateless — any CNode serves any client. Failover is instant and transparent. Zero data loss.

MODULE 2 — Storage Performance Fundamentals

Architecture Diagram: Performance Triangle



Scenario 1: Baseline Performance — Sequential R/W

PURPOSE: Measure raw read and write throughput. This is your baseline number before any tuning.

fio Option	Value	Meaning
--rw	write or read	Direction of I/O
--bs	1M	Block size: 1 Megabyte per operation
--size	256M	Total data written/read
--numjobs	1	One parallel worker
--group_reporting	(flag)	Show combined stats
2>&1 grep bw=	(pipe)	Show only bandwidth line

```
echo "=== Sequential Write ==="
# Write 256MB sequentially with 1MB blocks
# Measures: how fast can we write a large file
fio --name=seqwrite --rw=write --bs=1M --size=256M \
    --directory=/mnt/client1 --numjobs=1 --group_reporting \
    2>&1 | grep -E "bw=|iops="

echo "=== Sequential Read ==="
# Read back 256MB - measures read throughput
fio --name=seqread --rw=read --bs=1M --size=256M \
    --directory=/mnt/client1 --numjobs=1 --group_reporting \
    2>&1 | grep -E "bw=|iops="
```

✓ EXPECTED OUTPUT: WRITE: bw=4130MB/s | READ: bw=2256MB/s

Scenario 2: Block Size Effect on Throughput

PURPOSE: Show how block size changes throughput. Students find optimal block size for their workload.

```
# Test 4 block sizes and compare throughput
# 4k = database block size - many small ops
# 64k = medium block - mixed workloads
# 256k = large block - analytics
```

```
# 1m = maximum block - AI training
for bs in 4k 64k 256k 1m; do
  echo "=== Block size: $bs ==="
  fio --name=bs_$bs --rw=write --bs=$bs --size=128M \
    --directory=/mnt/client1 --numjobs=1 --group_reporting \
    2>&1 | grep -E "bw=|iops="
done
```

✓ EXPECTED OUTPUT: 4k: 767MB/s | 64k: 2397MB/s | 256k: 2739MB/s | 1m: 2739MB/s

Distributed System Challenge: Parallel Writers

🔧 **FIX APPLIED: FIX 1 APPLIED:** Removed duplicate code block. Only the verified one-liner version is kept below.

🎯 **PURPOSE:** Show how parallel writers scale throughput. Key differentiator between NAS and VAST.

```
# Run 1, 4, and 8 parallel jobs - compare total throughput each time
# numjobs=1 = single client
# numjobs=4 = 4 clients writing simultaneously
# numjobs=8 = 8 clients writing simultaneously
echo "=== 1 Job ===" && fio --name=j1 --rw=write --bs=1M --size=256M \
  --directory=/mnt/client1 --numjobs=1 --group_reporting 2>&1 | grep bw=

echo "=== 4 Jobs ===" && fio --name=j4 --rw=write --bs=1M --size=256M \
  --directory=/mnt/client1 --numjobs=4 --group_reporting 2>&1 | grep bw=

echo "=== 8 Jobs ===" && fio --name=j8 --rw=write --bs=1M --size=256M \
  --directory=/mnt/client1 --numjobs=8 --group_reporting 2>&1 | grep bw=
```

✓ EXPECTED OUTPUT: 1 Job: 2918MB/s | 4 Jobs: 6669MB/s | 8 Jobs: 1022MB/s (local disk limit)

💡 **KEY LESSON:** 1→4 jobs: throughput doubles. 8 jobs: loopback hits local disk saturation. On real VAST cluster, 8 jobs continue scaling linearly — no controller bottleneck.

MODULE 3 — Network Performance Analysis

Architecture Diagram: nconnect

Without nconnect (default): Client —[1 TCP]→ CNode	With nconnect=8: Client —[8 TCP]→ CNode —[8 TCP]→
One pipe = bandwidth capped	8 pipes = full bandwidth

nconnect is the single highest-impact NFS tuning option.
Default = 1 connection. VAST recommendation = 8-16.

Scenario 1: Bandwidth Test (loopback)

🎯 **PURPOSE:** Measure raw network bandwidth. Establishes network baseline before NFS testing.

```
# Start iperf3 server in background (-D = daemon mode)
iperf3 -s -D

# Wait 1 second for server to be ready
sleep 1

# Run client test:
# -c 127.0.0.1 = connect to loopback
# -t 10        = run for 10 seconds
# -P 4         = use 4 parallel streams
iperf3 -c 127.0.0.1 -t 10 -P 4
```

✅ **EXPECTED OUTPUT:** [SUM] 0.00-10.00 sec XX GBytes XX Gbits/sec sender

Scenario 2: nconnect — Single vs Multi Connection

🎯 **PURPOSE:** Prove that nconnect=8 delivers more throughput than nconnect=1.

```
echo "=== Connection count ==="
# Count active TCP connections to NFS port 2049
ss -tn dst :2049 | wc -l | xargs echo "Total NFS connections:"

echo "=== Single connection (nconnect=1) ==="
# /mnt/single was mounted with nconnect=1 - only 1 TCP pipe
fio --name=single --rw=write --bs=1M --size=256M \
    --directory=/mnt/single --numjobs=1 --group_reporting \
    2>&1 | grep bw=

echo "=== Multi connection (nconnect=8) ==="
# /mnt/multi was mounted with nconnect=8 - 8 TCP pipes
fio --name=multi --rw=write --bs=1M --size=256M \
    --directory=/mnt/multi --numjobs=1 --group_reporting \
    2>&1 | grep bw=
```

✅ **EXPECTED OUTPUT:** single: lower MB/s | multi: higher MB/s (nconnect improvement visible)

Distributed System Challenge: Network Delay Impact

🔧 **FIX APPLIED: FIX 2 APPLIED:** Added safety tc clear before add. Without this, running twice causes 'File exists' error.

🎯 **PURPOSE:** Inject 20ms delay and measure impact. Simulates slow network path to one CNode.

```
# SAFETY FIRST: remove any existing tc rule before adding new one
# 2>/dev/null = suppress error if no rule exists
sudo tc qdisc del dev lo root 2>/dev/null; echo "tc cleared"
```

```
echo "=== Before delay ==="
```

```
# Baseline performance with no delay
fio --name=before --rw=write --bs=1M --size=128M \
    --directory=/mnt/client1 --numjobs=1 --group_reporting \
    2>&1 | grep -E "bw=|lat="
```

```
echo "=== Injecting 20ms delay ==="
```

```
# tc = traffic control
# qdisc add = add queuing discipline
# dev lo = on loopback interface
# root = at root level (all traffic)
# netem = network emulator
# delay 20ms = add 20ms to every packet
sudo tc qdisc add dev lo root netem delay 20ms
```

```
echo "=== After delay ==="
```

```
# Same test - throughput lower, latency higher
fio --name=after --rw=write --bs=1M --size=128M \
    --directory=/mnt/client1 --numjobs=1 --group_reporting \
    2>&1 | grep -E "bw=|lat="
```

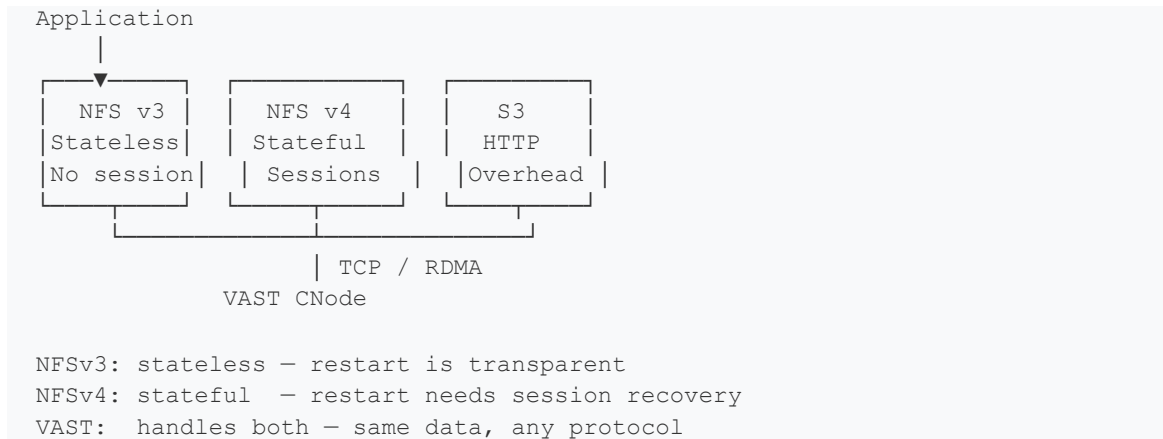
```
# ALWAYS remove delay after test - clean up
sudo tc qdisc del dev lo root && echo "Network restored"
```

✅ **EXPECTED OUTPUT:** tc cleared | Before: high MB/s | After: lower MB/s, higher lat | Network restored

💡 **KEY LESSON:** Network delay impacts only that client's CNode path. Other clients on other CNodes are unaffected. This is DASE isolation — per-client network paths are independent.

MODULE 4 — Protocol Performance Analysis

Architecture Diagram: Protocol Stack



Scenario 1: NFSv3 vs NFSv4 Write Performance

🎯 **PURPOSE:** Compare NFSv3 and NFSv4 throughput. Shows protocol overhead difference.

```
echo "=== NFSv3 Write ==="
# /mnt/v3 was mounted with -o vers=3 (stateless)
# NFSv3 = simpler protocol, less overhead
fio --name=v3write --rw=write --bs=1M --size=256M \
    --directory=/mnt/v3 --numjobs=1 --group_reporting \
    2>&1 | grep -E "bw=|iops="

echo "=== NFSv4 Write ==="
# /mnt/v4 was mounted with -o vers=4 (stateful)
# NFSv4 = more features but more protocol overhead
fio --name=v4write --rw=write --bs=1M --size=256M \
    --directory=/mnt/v4 --numjobs=1 --group_reporting \
    2>&1 | grep -E "bw=|iops="
```

✅ **EXPECTED OUTPUT:** v3: bw=XXXX MB/s | v4: bw=XXXX MB/s (compare the two numbers)

Scenario 2: Watch NFS Operations Before and After Workload

🎯 **PURPOSE:** Show what NFS operations fire during normal file work. Makes the invisible protocol visible.

NFS Operation	Triggered By	What It Does
read	cat, cp source	Read file data from server
write	cp dest, dd	Write file data to server
getattr	ls -la, stat	Get file metadata
lookup	any path access	Find file by name
readdir	ls	List directory contents
create	touch, cp new file	Create new file

```
echo "=== Before workload ==="
# Capture NFS operation counters BEFORE workload
nfsstat -c | head -8
```

```

echo "=== Running workload ==="
# Copy 30 files, list dir, read each back
# Triggers: create, write, readdir, read, getattr, lookup
for i in $(seq 1 30); do
    cp /etc/passwd /mnt/client1/file_$i 2>/dev/null
    ls /mnt/client1/ > /dev/null
    cat /mnt/client1/file_$i > /dev/null
done

echo "=== After workload ==="
# Compare counters - read, write, readdir counts jumped
nfsstat -c | head -8

echo "=== Cleanup ==="
rm -f /mnt/client1/file_*

```

✓ EXPECTED OUTPUT: Before: calls=878 | After: calls=1200+ (counters visibly increased)

Distributed System Challenge: Session Recovery After Restart

🎯 PURPOSE: Prove data written before server restart is accessible after — data durability guarantee.

```

# Write a file before server restart
echo "data before restart" > /mnt/client1/session_test.txt
cat /mnt/client1/session_test.txt

echo "=== Restarting NFS server ==="
sudo systemctl restart nfs-kernel-server
sleep 3

echo "=== Access after restart ==="
# File must still be readable - data was persisted to disk
cat /mnt/client1/session_test.txt && echo "File accessible after restart"
ls /mnt/client1/ | head -5
rm -f /mnt/client1/session_test.txt

```

✓ EXPECTED OUTPUT: data before restart | File accessible after restart

💡 KEY LESSON: NFSv3 = stateless = transparent after restart. NFSv4 = stateful = needs session recovery. VAST handles both automatically. Data is always persisted to QLC flash before client acknowledgement.

MODULE 5 — Performance Optimization

Architecture Diagram: Optimization Layers

Layer 5: Application — parallel I/O, large files
↓
Layer 4: Mount Options — nconnect, rsize, wsize, noatime
↓
Layer 3: Network — MTU 9000, RDMA, bonding
↓
Layer 2: OS Kernel — dirty_ratio, readahead
↓
Layer 1: Storage — QoS, data reduction policy

Lab: we tune Layer 4 (mount options) and Layer 2 (kernel).
Real VAST: all 5 layers are tunable via vms commands.

Scenario 1: Baseline vs Tuned vs Optimized

🎯 **PURPOSE:** Prove mount options make a measurable difference. Students see numbers change with their own commands.

Mount	Key Options	Expected Throughput
/mnt/baseline	default (1 conn, 256KB rsize)	Lowest
/mnt/tuned	nconnect=8, rsize/wsize=1MB	Higher
/mnt/optimized	nconnect=16, noatime, 1MB	Highest

```
echo "=== BASELINE (default -- untuned) ==="
# 1 TCP connection, default 256KB block size
fio --name=base --rw=write --bs=1M --size=256M \
    --directory=/mnt/baseline --numjobs=1 --group_reporting \
    2>&1 | grep bw=

echo "=== TUNED (nconnect=8 + rsize/wsize=1MB) ==="
# 8 TCP connections + 1MB read/write block size
fio --name=tuned --rw=write --bs=1M --size=256M \
    --directory=/mnt/tuned --numjobs=1 --group_reporting \
    2>&1 | grep bw=

echo "=== OPTIMIZED (nconnect=16 + noatime) ==="
# 16 TCP connections + skip access time updates
fio --name=opt --rw=write --bs=1M --size=256M \
    --directory=/mnt/optimized --numjobs=1 --group_reporting \
    2>&1 | grep bw=
```

✅ **EXPECTED OUTPUT:** baseline: X MB/s | tuned: higher | optimized: highest

Scenario 2: Kernel Dirty Ratio Tuning

🎯 **PURPOSE:** Show how Linux kernel write buffering affects NFS throughput.

```
echo "=== Current kernel settings ==="
# dirty_ratio: % of RAM that can be dirty before forced flush
# Lower value = more frequent flushes = lower throughput
# Higher value = less frequent flushes = higher throughput
cat /proc/sys/vm/dirty_ratio
```



```
cat /proc/sys/vm/dirty_background_ratio

echo "=== Before tuning ==="
fio --name=before --rw=write --bs=1M --size=256M \
    --directory=/mnt/client1 --numjobs=1 --group_reporting \
    2>&1 | grep bw=

echo "=== Apply tuning ==="
# Allow 40% of RAM as dirty pages = fewer interruptions
sudo sysctl -w vm.dirty_ratio=40
sudo sysctl -w vm.dirty_background_ratio=10

echo "=== After tuning ==="
fio --name=after --rw=write --bs=1M --size=256M \
    --directory=/mnt/client1 --numjobs=1 --group_reporting \
    2>&1 | grep bw=

echo "=== Restore defaults ==="
sudo sysctl -w vm.dirty_ratio=20
sudo sysctl -w vm.dirty_background_ratio=5
```

✓ EXPECTED OUTPUT: before: X MB/s | after tuning: $X+N$ MB/s | settings restored

Distributed System Challenge: Noisy Neighbour

🎯 **PURPOSE:** Show that without QoS, a greedy writer degrades a sensitive reader's latency.

```
echo "=== Start greedy writer ==="
# 8 parallel jobs writing 512MB - consumes maximum bandwidth
# & puts it in background so next command runs simultaneously
fio --name=greedy --rw=write --bs=1M --size=512M \
    --directory=/mnt/client1 --numjobs=8 --group_reporting &

# Wait 2 seconds for greedy writer to start consuming bandwidth
sleep 2

echo "=== Sensitive reader competing ==="
# 1 job, small 4K reads - latency-sensitive
# Observe: latency is higher because greedy writer competes
fio --name=sensitive --rw=read --bs=4k --size=128M \
    --directory=/mnt/optimized --numjobs=1 --group_reporting \
    2>&1 | grep -E "bw=|lat="

# Wait for greedy writer background job to finish
wait
echo "Both workloads complete"
```

✓ EXPECTED OUTPUT: sensitive reader shows higher latency while greedy writer runs

💡 **KEY LESSON:** Without QoS, greedy workloads steal bandwidth from sensitive ones. VAST QoS policy per view guarantees each tenant minimum performance regardless of neighbours.

MODULE 6 — Storage Service Troubleshooting

Architecture Diagram: Triage Path

```
'Storage not working' complaint
|
Step 1: Network OK?    → ping -c 3 127.0.0.1
|
Step 2: Port open?     → nc -zv 127.0.0.1 2049
|
Step 3: Mount present? → findmnt -t nfs,nfs4
|
Step 4: Can write?     → touch /mnt/client1/.test
|
Step 5: RPC up?        → rpcinfo -p 127.0.0.1
|
Step 6: Exports OK?   → showmount -e 127.0.0.1
|
VAST cluster: vms cluster health

Stop at first failure — that is your problem.
```

Scenario 1: Full Triage Checklist

 **PURPOSE:** Run complete triage top-to-bottom. Stop at the first step that fails.

```
echo "=== Step 1: Network reachability ==="
# Can we reach the server at all?
ping -c 3 127.0.0.1 | tail -2

echo "=== Step 2: NFS port check ==="
# Is NFS port 2049 accepting connections?
nc -zv 127.0.0.1 2049 && echo "Port 2049 OPEN"
# Is RPC portmapper on port 111 working?
nc -zv 127.0.0.1 111 && echo "Port 111 OPEN"

echo "=== Step 3: Mounts present ==="
findmnt -t nfs,nfs4 -o TARGET,SOURCE

echo "=== Step 4: Write test ==="
# Most common failure point — can we actually write?
touch /mnt/client1/.triage && echo "Write OK" && rm /mnt/client1/.triage

echo "=== Step 5: RPC services ==="
rpcinfo -p 127.0.0.1 | head -8

echo "=== Step 6: Exports ==="
showmount -e 127.0.0.1
```

 **EXPECTED OUTPUT:** All steps return OK — system healthy

Scenario 2: Simulate and Recover Stale Mount

 **PURPOSE:** Practice the most common real-world NFS issue: stale mount after server restart.

```

echo "=== Stopping NFS server ==="
# Simulate CNode going down
sudo systemctl stop nfs-kernel-server
sleep 2

echo "=== Testing stale mount ==="
# timeout 5 = give up after 5 seconds (prevents forever hang)
# || means: run echo if timeout command fails
timeout 5 ls /mnt/client1 || echo "Mount is stale - as expected"

echo "=== Force unmount ==="
# -f = force even if server unreachable
# -l = lazy: detach now, clean up when idle
sudo umount -f -l /mnt/client1

echo "=== Restarting server ==="
sudo systemctl start nfs-kernel-server
sleep 2

echo "=== Remounting ==="
sudo mount -t nfs 127.0.0.1:/srv/nfs/vol1 /mnt/client1

echo "=== Verify recovery ==="
ls /mnt/client1 && echo "Mount recovered successfully"

```

✓ EXPECTED OUTPUT: Mount is stale — as expected | Mount recovered successfully

Distributed System Challenge: Permission Failure Triage

🎯 **PURPOSE:** Diagnose and fix permission failures — second most common real-world NFS issue.

⚠️ **NOTE:** ALWAYS fix permissions at the end of this exercise. If not fixed, all subsequent modules fail.

```

echo "=== Breaking permissions ==="
sudo chmod 700 /srv/nfs/vol1
sudo chown root:root /srv/nfs/vol1

echo "=== Observe failure ==="
# 2>&1 redirects error to terminal so we can see it
touch /mnt/client1/test 2>&1 || echo "Permission denied - as expected"

echo "=== Triage ==="
# Check who owns the server-side directory
ls -ld /srv/nfs/vol1
# Check who we are
id

echo "=== Fix permissions ==="
sudo chown $USER:$USER /srv/nfs/vol1
sudo chmod 755 /srv/nfs/vol1

echo "=== Verify fix ==="
touch /mnt/client1/test && echo "Fixed - write OK"
rm -f /mnt/client1/test

```

✓ EXPECTED OUTPUT: Permission denied — as expected | Fixed — write OK

 **FIX APPLIED: FIX 3 APPLIED:** After Scenario 2, only /mnt/client1 was remounted. Other 8 mounts were missing. Quick Reset below restores all mounts before Module 7.

Run Quick Reset After Module 6

 **NOTE:** Run this after completing Module 6. Scenario 2 stops the NFS server — other mounts need restoring.

```
# Restore all mounts after Module 6 exercises
sudo umount -f -l /mnt/voll /mnt/v3 /mnt/v4 /mnt/baseline \
  /mnt/tuned /mnt/optimized /mnt/multi /mnt/single 2>/dev/null
sudo systemctl restart nfs-kernel-server
sleep 2
sudo chown $USER:$USER /srv/nfs/voll /srv/nfs/aidata /srv/nfs/scratch
sudo chmod 755 /srv/nfs/voll
sudo mount -t nfs 127.0.0.1:/srv/nfs/voll /mnt/client1
sudo mount -t nfs 127.0.0.1:/srv/nfs/voll /mnt/voll
sudo mount -t nfs -o vers=3 127.0.0.1:/srv/nfs/voll /mnt/v3
sudo mount -t nfs -o vers=4 127.0.0.1:/srv/nfs/voll /mnt/v4
sudo mount -t nfs 127.0.0.1:/srv/nfs/voll /mnt/baseline
sudo mount -t nfs -o nconnect=8,rsiz=1048576,wsiz=1048576 \
  127.0.0.1:/srv/nfs/voll /mnt/tuned
sudo mount -t nfs -o
vers=3,nconnect=16,rsiz=1048576,wsiz=1048576,noatime,hard,proto=tcp \
  127.0.0.1:/srv/nfs/voll /mnt/optimized
sudo mount -t nfs -o vers=3,nconnect=8 127.0.0.1:/srv/nfs/voll /mnt/multi
sudo mount -t nfs -o vers=3,nconnect=1 127.0.0.1:/srv/nfs/voll /mnt/single
findmnt -t nfs,nfs4 -o TARGET,SOURCE && echo "ALL MOUNTS RESTORED"
```

 **EXPECTED OUTPUT:** 9 mounts listed | ALL MOUNTS RESTORED

 **KEY LESSON:** Triage order: network → port → mount → write → permissions → server. Fix is always server-side for permission errors. Always run Quick Reset after Module 6.

MODULE 7 — Performance Troubleshooting

Architecture Diagram: Troubleshooting Decision Tree

```
Performance complaint
|
nfsstat -c | head -6
retrans > 0?
YES → Network problem → check tc, ping, iperf3
NO → Continue
|
ss -tn dst :2049 | wc -l
connections = 1?
YES → Add nconnect → remount with nconnect=8
NO → Continue
|
fio --iodepth=1 vs --iodepth=32
Low IOPS at depth 1?
YES → Increase iodepth in application
NO → Hardware or network issue
```

Scenario 1: Step-by-Step Throughput Diagnosis

🎯 **PURPOSE:** Walk through the standard diagnosis path — see performance improve at each step.

```
echo "=== Step 1: Current throughput ==="
# Start with baseline (untuned) mount
fio --name=diag --rw=write --bs=1M --size=256M \
    --directory=/mnt/baseline --numjobs=1 --group_reporting \
    2>&1 | grep bw=

echo "=== Step 2: Retransmit check ==="
# retrans column MUST be 0 - any retrans = network problem
nfsstat -c | head -6

echo "=== Step 3: Connection count ==="
# How many TCP connections to NFS port?
ss -tn dst :2049 | wc -l | xargs echo "Active connections:"

echo "=== Step 4: Tuned throughput ==="
fio --name=tuned --rw=write --bs=1M --size=256M \
    --directory=/mnt/tuned --numjobs=1 --group_reporting \
    2>&1 | grep bw=

echo "=== Step 5: Optimized throughput ==="
fio --name=opt --rw=write --bs=1M --size=256M \
    --directory=/mnt/optimized --numjobs=1 --group_reporting \
    2>&1 | grep bw=
```

✅ **EXPECTED OUTPUT:** baseline: X MB/s | retrans=0 | tuned: higher | optimized: highest

Scenario 2: Queue Depth Impact on IOPS

🎯 **PURPOSE:** Show that increasing I/O queue depth dramatically improves IOPS for random workloads.

```

echo "=== Queue depth 1 (low) ==="
# iodepth=1: only 1 I/O request in flight at a time
# Like single-lane road — one car at a time
fio --name=qd1 --rw=randread --bs=4k --size=128M \
    --directory=/mnt/client1 --iodepth=1 --numjobs=1 \
    --group_reporting 2>&1 | grep -E "iops=|lat="

echo "=== Queue depth 16 (high) ==="
# iodepth=16: 16 requests in flight at once
# Like 16-lane highway — many cars at once
fio --name=qd16 --rw=randread --bs=4k --size=128M \
    --directory=/mnt/client1 --iodepth=16 --numjobs=4 \
    --group_reporting 2>&1 | grep -E "iops=|lat="

```

✓ EXPECTED OUTPUT: *qd1: XXXX IOPS | qd16: XXXX IOPS (significantly higher with depth 16)*

Distributed System Challenge: Inject and Detect Latency

🔧 **FIX APPLIED: FIX 2 APPLIED (same as Module 3): Added safety to clear before add.**

🎯 **PURPOSE:** Inject delay, measure impact, remove, verify recovery. Systematic latency diagnosis.

```

# SAFETY: remove any existing tc rule first
sudo tc qdisc del dev lo root 2>/dev/null; echo "tc cleared"

echo "=== Baseline latency ==="
fio --name=base --rw=randread --bs=4k --size=128M \
    --directory=/mnt/client1 --numjobs=1 --group_reporting \
    2>&1 | grep -E "bw=|lat="

echo "=== Inject 20ms delay ==="
sudo tc qdisc add dev lo root netem delay 20ms

echo "=== Latency under delay ==="
fio --name=delayed --rw=randread --bs=4k --size=128M \
    --directory=/mnt/client1 --numjobs=1 --group_reporting \
    2>&1 | grep -E "bw=|lat="

echo "=== Remove delay ==="
sudo tc qdisc del dev lo root

echo "=== Verify restored ==="
fio --name=restored --rw=randread --bs=4k --size=128M \
    --directory=/mnt/client1 --numjobs=1 --group_reporting \
    2>&1 | grep -E "bw=|lat="

```

✓ EXPECTED OUTPUT: *tc cleared | baseline: low lat | delayed: high lat | restored: low lat*

💡 **KEY LESSON:** Retransmits > 0 = network problem. Latency spike = overload. Systematic approach: baseline → inject fault → measure → fix → verify recovery.

MODULE 8 — Log Analysis and Diagnostics

Architecture Diagram: Log Sources

Ubuntu Client Side:

```
dmesg -T
└─ kernel NFS messages

/var/log/syslog
└─ system events

journalctl
└─ systemd services
```

VAST Cluster Side:

```
vms events list
vms alerts list
vms audits list
vms supportbundles create
vms cnodes logs --id 1
```

On Ubuntu lab: dmesg is primary source.
journalctl -u nfs-kernel-server may return empty – use dmesg.

Scenario 1: Build Incident Timeline

 **FIX APPLIED: FIX 4 APPLIED: Replaced journalctl (returned empty in live session) with dmesg -T (verified working).**

 **PURPOSE: Trigger a real incident and build a timeline from logs. Practice reading kernel messages.**

```
echo "=== Trigger incident ==="
# Stop and restart NFS server – creates real log entries
sudo systemctl stop nfs-kernel-server
sleep 5
sudo systemctl start nfs-kernel-server
sleep 2

echo "=== Kernel log – NFS messages ==="
# dmesg = kernel ring buffer (last N kernel messages)
# -T     = human-readable timestamps
# grep -iE = case-insensitive extended regex
dmesg -T | grep -iE "nfs|error|fail" | tail -10

echo "=== All NFS messages ==="
dmesg -T | grep -i nfs | tail -15

echo "=== Recent errors in kernel ==="
dmesg -T | grep -iE "error|fail|timeout" | tail -10
```

 **EXPECTED OUTPUT:** Kernel messages with timestamps showing NFS stop/start events

Scenario 2: Log Correlation Across Nodes

 **PURPOSE: Simulate logs from multiple CNodes and find which one had the error. Real daily VAST admin task.**

```
# Create directory for simulated node logs
mkdir -p /tmp/fakelogs

# Create normal log entry for each CNode
```

```
# > (single) = overwrite (create fresh)
for node in cnode1 cnode2 cnode3; do
    echo "$(date) INFO $node: serving client 10.0.0.50" > /tmp/fakelogs/$node.log
done

# Inject error into cnode2 only
# >> (double) = append to existing file
echo "$(date) ERROR cnode2: NVMe timeout on dbx3" >> /tmp/fakelogs/cnode2.log
echo "$(date) ERROR cnode2: write failed client 10.0.0.50" >>
/tmp/fakelogs/cnode2.log

echo "=== Which node has error? ==="
# -l = print only filenames that match
grep -l "ERROR" /tmp/fakelogs/*.log

echo "=== Error details ==="
grep "ERROR" /tmp/fakelogs/*.log

echo "=== Merged timeline ==="
# Combine all logs and sort by timestamp
cat /tmp/fakelogs/*.log | sort
```

✓ EXPECTED OUTPUT: /tmp/fakelogs/cnode2.log | 2 ERROR lines | Sorted timeline — cnode2 identified

Distributed System Challenge: Pattern Detection

🎯 **PURPOSE:** Generate real log activity and extract meaningful patterns at scale.

```
echo "=== Generate log activity ==="
# Restart twice — creates start/stop entries in kernel log
sudo systemctl restart nfs-kernel-server
sleep 2
sudo systemctl restart nfs-kernel-server
sleep 2

echo "=== NFS specific messages ==="
dmesg -T | grep -i nfs | tail -15

echo "=== Error count ==="
# Count total error-type messages in kernel log
dmesg -T | grep -iE "error|fail|timeout" | wc -l | xargs echo "Total errors:"

echo "=== Recent kernel events ==="
# Last 20 kernel messages — what happened recently
dmesg -T | tail -20
```

✓ EXPECTED OUTPUT: NFS messages with timestamps | Total errors: N | Recent kernel events

💡 **KEY LESSON:** Always use `dmesg -T` on Ubuntu — `journalctl` NFS service logs may be empty. Key pattern: multiple restarts in short time = instability. Zero errors after restart = clean recovery.

MODULE 9 — Monitoring Fundamentals

Architecture Diagram: Monitor These 5 Things Always

1. Cluster Health → any component FAILED?
Ubuntu: findmnt loop + write test
VAST: vms cluster health
2. Capacity → usage > 80%?
Ubuntu: df /mnt/client1 | awk '{print \$5}'
VAST: vms cluster capacity
3. Throughput → below baseline?
Ubuntu: fio periodic + log to file
VAST: vms metrics get --metric throughput
4. Latency → above threshold?
Ubuntu: nfsiostat, mountstats
VAST: vms metrics get --metric latency
5. Error Rate → retransmits > 0?
Ubuntu: nfsstat -c | awk 'NR==4{print \$2}'
VAST: vms alerts list --state ACTIVE

Scenario 1: Live Dashboard

 **PURPOSE:** Auto-refreshing dashboard showing all key metrics in one view.

 **NOTE:** Press **Ctrl+C** to exit watch. If terminal appears frozen — just press **Ctrl+C**.

```
# watch = run a command repeatedly and show output
# -n3    = refresh every 3 seconds
# Everything inside single quotes is the repeated command
watch -n3 '
echo "=== VAST Admin Dashboard $(date) ==="
echo ""
echo "--- NFS Mounts ---"
findmnt -t nfs,nfs4 -o TARGET,SOURCE
echo ""
echo "--- Capacity ---"
df -h /mnt/client1
echo ""
echo "--- NFS Stats ---"
nfsstat -c 2>/dev/null | head -5
echo ""
echo "--- Connections ---"
ss -tn dst :2049 | wc -l | xargs echo "Active NFS connections:"
echo ""
echo "--- Recent Errors ---"
dmesg -T | grep -iE "error|fail" | tail -3 || echo "None"
'
```

 **EXPECTED OUTPUT:** Live dashboard refreshing every 3s — press **Ctrl+C** to exit

Scenario 2: Full Health Check + Capacity Alert

🎯 **PURPOSE:** Automated health check returning clear PASS/FAIL for each metric.

```
echo "=== Mount health ==="
# Check all 9 training mounts are present
for mnt in client1 vol1 v3 v4 baseline tuned optimized multi single; do
    # findmnt returns 0=found, non-zero=missing
    findmnt /mnt/$mnt > /dev/null 2>&1 \
        && echo "✓ /mnt/$mnt" \
        || echo "✗ /mnt/$mnt MISSING"
done

echo ""
echo "=== Write health ==="
# Create and remove test file - confirms write works
touch /mnt/client1/.hc && echo "✓ Write OK" && rm /mnt/client1/.hc

echo ""
echo "=== Retransmit check ==="
# NR==4 = 4th line of nfsstat output
# $2 = second column = retransmit count
RETRANS=$(nfsstat -c 2>/dev/null | awk 'NR==4{print $2}')
[ "$RETRANS" = "0" ] \
    && echo "✓ Retransmits: 0 - healthy" \
    || echo "✗ WARNING retransmits=$RETRANS"

echo ""
echo "=== Capacity check ==="
# $5 = 5th column of df = use percentage
# tr -d '%' = remove percent sign for numeric comparison
USED=$(df /mnt/client1 | awk 'NR==2{print $5}' | tr -d '%')
[ "$USED" -lt 80 ] \
    && echo "✓ Capacity OK at ${USED}%" \
    || echo "✗ ALERT capacity at ${USED}%"
```

✓ **EXPECTED OUTPUT:** ✓ 9 mounts | ✓ Write OK | ✓ Retransmits: 0 | ✓ Capacity 20% — OK

Distributed System Challenge: Aggregate Cluster Health

🎯 **PURPOSE:** Full cluster health check with final GREEN/ALERT verdict. The VAST admin morning routine.

```
echo "=== Full Cluster Health Check ==="
# Track overall state - starts at 0 (healthy)
ALLFAIL=0

# Check all 9 mounts
for mnt in client1 vol1 v3 v4 baseline tuned optimized multi single; do
    if findmnt /mnt/$mnt > /dev/null 2>&1; then
        echo "✓ /mnt/$mnt"
    else
        echo "✗ /mnt/$mnt MISSING"
        ALLFAIL=1 # flag set if any mount missing
    fi
done

# Check retransmits
RETRANS=$(nfsstat -c 2>/dev/null | awk 'NR==4{print $2}')
```

```

[ "$RETRANS" = "0" ] && echo "✓ Retransmits: 0" || echo "✗
retransmits=$RETRANS"

# Check capacity
USED=$(df /mnt/client1 | awk 'NR==2{print $5}' | tr -d '%')
[ "$USED" -lt 80 ] && echo "✓ Capacity ${USED}%" || echo "✗ Capacity ${USED}%
ALERT"

echo ""
# Final verdict based on ALLFAIL flag
if [ $ALLFAIL -eq 0 ]; then
    echo "=====
    echo " ALL GREEN — Cluster healthy"
    echo "=====
else
    echo "=====
    echo " ALERT — Check failed mounts"
    echo "=====
fi

```

✓ EXPECTED OUTPUT: 9 green ticks | ✓ Retransmits: 0 | ✓ Capacity 20% | ALL GREEN — Cluster healthy

💡 **KEY LESSON:** With 50 clients: monitor aggregates not individuals. Any retrans > 0 = investigate. Capacity > 80% = alert. The morning health check is the most important VAST admin habit.

QUICK RESET — Emergency Restore All Mounts

🎯 **PURPOSE:** Run any time mounts break, permissions are wrong, or tc delay is stuck. Restores everything to clean state.

⚠️ **NOTE:** Always run this after Module 6 exercises — that module intentionally breaks permissions and mounts.

```
# Force unmount all training mounts (ignore errors if already unmounted)
sudo umount -f -l /mnt/client1 /mnt/voll /mnt/v3 /mnt/v4 \
  /mnt/baseline /mnt/tuned /mnt/optimized /mnt/multi /mnt/single 2>/dev/null

# Restart NFS server cleanly
sudo systemctl restart nfs-kernel-server
sleep 2

# Fix permissions (in case Module 6 broke them)
sudo chown $USER:$USER /srv/nfs/voll /srv/nfs/aidata /srv/nfs/scratch
sudo chmod 755 /srv/nfs/voll

# Remove any tc delay left over from Modules 3 or 7
sudo tc qdisc del dev lo root 2>/dev/null

# Remount all 9 training mounts
sudo mount -t nfs 127.0.0.1:/srv/nfs/voll /mnt/client1
sudo mount -t nfs 127.0.0.1:/srv/nfs/voll /mnt/voll
sudo mount -t nfs -o vers=3 127.0.0.1:/srv/nfs/voll /mnt/v3
sudo mount -t nfs -o vers=4 127.0.0.1:/srv/nfs/voll /mnt/v4
sudo mount -t nfs 127.0.0.1:/srv/nfs/voll /mnt/baseline
sudo mount -t nfs -o nconnect=8,rsiz=1048576,wsiz=1048576 \
  127.0.0.1:/srv/nfs/voll /mnt/tuned
sudo mount -t nfs \
  -o vers=3,nconnect=16,rsiz=1048576,wsiz=1048576,noatime,hard,proto=tcp \
  127.0.0.1:/srv/nfs/voll /mnt/optimized
sudo mount -t nfs -o vers=3,nconnect=8 127.0.0.1:/srv/nfs/voll /mnt/multi
sudo mount -t nfs -o vers=3,nconnect=1 127.0.0.1:/srv/nfs/voll /mnt/single

# Verify all 9 mounts restored
findmnt -t nfs,nfs4 -o TARGET,SOURCE && echo "ALL MOUNTS RESTORED"
```

✅ **EXPECTED OUTPUT:** 9 mounts listed | ALL MOUNTS RESTORED

Quick Reference — All 9 Modules

Module	Topic	Key Command	Watch For
1	Architecture	showmount -e, findmnt, df -h	All 4 exports and 9 mounts visible
2	Performance	fio --rw=write --bs=1M	bw= increases with jobs 1→4
3	Network	iperf3, tc qdisc netem	Clear tc BEFORE add — always
4	Protocol	nfsstat -c, fio /mnt/v3 vs v4	retrans = 0 always
5	Optimization	fio baseline vs tuned vs optimized	Throughput increases each step
6	Troubleshooting	timeout, umount -f -l	Run Quick Reset after this module
7	Perf Debug	tc netem, fio iodepth	Clear tc BEFORE add — always
8	Log Analysis	dmesg -T, grep ERROR	Use dmesg not journalctl
9	Monitoring	watch, findmnt loop, df	Ctrl+C to exit watch